

Caleb Bui
134001202

Task 0: Lab preparation

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec$ python3 --version
Python 3.12.3
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec$ pip show cryptography pytest
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors <cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
---
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laughner, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec$

ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ openssl dhparam -in ffdhe3072.pem -text -noout | head -3
DH Parameters: (3072 bit)
GROUP: ffdhe3072
ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
ubuntu@ubuntu2204:~/csce465-agentsec/hw2$
```

Task 1:

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec$ python3 hw2/ctr_baseline.py
Original plaintext:
{"action":"READ","path":"notes.txt"}

Ciphertext:
7d3379ab007611fc8ad97e09ded2205fcac12fa2d5bf9ba98d1ffffc474fb4e6f7e5870b

XOR relation between original and modified bytes:
Original:  b'READ'
Modified:  b'LIST'
52 XOR 4c = 1e
45 XOR 49 = 0c
41 XOR 53 = 12
44 XOR 54 = 10

Modified plaintext:
{"action":"LIST","path":"notes.txt"}

Replay demonstration:
First time receiver processes:
{"action":"READ","path":"notes.txt"}

Second time receiver processes the SAME ciphertext:
{"action":"READ","path":"notes.txt"}
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec$
```

The python program is a simple system that encrypts a fixed command using AES-CTR without a MAC. A relay sits between the sender and receiver and can modify or replay the encrypted command.

The relay changed READ to LIST without knowing the AES key by using the XOR relationship between the original and modified bytes. The receiver decrypted the modified ciphertext as LIST, and when the original ciphertext was sent again, the receiver processed it a second time because there was nothing to identify it as a replay

The root cause is that AES-CTR provides confidentiality, but not integrity or replay protection. It hides the contents of the message, but without a MAC or authentication mechanism, the receiver cannot tell whether the ciphertext was modified or whether the same message was already received and processed

Task 2:

The Gateway and Node use Diffie–Hellman to establish a shared secret using fresh private values and nonces for each session. Their long-term RSA-PSS keys authenticate the handshake by signing the transcript, which contains both identities, roles, nonces, and DH public values. The transcript hash is used with the shared secret to derive separate encryption

and MAC keys for each direction. The fresh DH values and nonces provide replay resistance and forward secrecy, while the separate KDF labels provide key separation.

Signing both identities and roles binds each party to the specific handshake and prevents an attacker from substituting a different identity or switching the message direction. Including the nonces and DH public values in the signed transcript also helps prevent basic man-in-the-middle and reflection attacks because changes will cause signature verification to fail.

Without the signatures, Diffie–Hellman can still create a shared secret, but there is no way to verify who the other party is. An attacker could perform both man-in-the-middle and reflection attacks by creating separate DH exchanges with the Gateway and Node and modifying their communication.

Task 3:

Nothing to report, file will be added in git

Task 4:

The attacker can listen to messages, change them, replay them, or send fake messages between the Gateway and Node. We assume the attacker does not have the private keys or session keys. The attacker can therefore interact with the communication channel but cannot directly create valid signatures or MACs without the required keys. This design does not protect against a device that is already compromised or an attacker who has stolen a private key. It also does not prevent denial-of-service attacks where an attacker simply blocks or drops messages.

AES-CTR uses the IV to create a stream of data that is combined with the plaintext to encrypt it. If the same IV is used twice with the same key, the same stream is reused, which can let an attacker learn information about the messages. This is why every message needs a unique IV when using CTR mode. The design uses the session ID and sequence number to create a different IV for each record. The sequence number starts at zero and increases for each message, which also allows the receiver to detect replayed or out-of-order records.

The receiver checks the HMAC before decrypting the message. This means that if an attacker changes the header, IV, or ciphertext, the receiver rejects the record instead of decrypting and using the modified data. This is important because an attacker should not be able to modify encrypted data and have the receiver process the result. Checking the HMAC first also means that plaintext is not released when authentication fails. The receiver only decrypts a record after confirming that the record came from someone with the correct MAC key and that the protected data has not been changed.

The reason independent encryption and MAC keys are used is to keep each key focused on one security purpose. Separate keys are also used for communication in each direction, so the Gateway-to-Node keys are different from the Node-to-Gateway keys. This prevents the same key from being reused for multiple security tasks and helps prevent a message from one direction from being accepted as a message in the other direction.

AES-CTR and HMAC use two separate steps: AES-CTR encrypts the message, and HMAC checks that the message was not changed. AES-GCM combines encryption and

authentication into one process, which can make it simpler to implement and reduces the amount of separate cryptographic code needed. However, AES-GCM still requires a unique nonce for every message. Both approaches can provide confidentiality and integrity when implemented correctly, but incorrect nonce management can still cause serious security problems.

A record can be correctly encrypted and authenticated but still contain a dangerous command. For example, an authorized user could send a command that deletes a file. The cryptographic system can verify that the command was not changed and came from an authorized party, but it cannot determine whether the requested action is safe or appropriate. The application therefore still needs additional authorization and safety checks before executing sensitive commands. This shows that encryption and authentication protect the communication channel, but they do not replace application-level security.